

Six Weeks of Safe Exploitation: Teaching Plan with Worked Examples

Sep 28, 2026 · @Dusty

How to use this plan

In six weeks, students go from "what is an equilibrium" to reproducing and critiquing the paper's central tables. Each week builds one layer the paper stands on, and every worked example can be checked by hand or by running code from the project.

Assumed rhythm: one 90-minute meeting plus about 3.5 hours of independent work per student per week. Senior DS/CS background: comfortable with Python, probability, and expected value; no game theory assumed.

Each week has the same shape:

1. **Concepts** — what the week adds to the story.
2. **Worked examples** — small enough to do on paper in the meeting, each with its answer. Most are toy versions of something the paper measures in full Leduc.
3. **Hands-on** — the same idea in the real game, using `rn timer/package/`.
4. **Reading and check questions** — for the week's independent work.

One-time setup (first meeting or before it):

```
cd rn timer/package
python tree_engine.py build      # about 2 minutes, writes leduc3p_tree.npz
python tree_engine.py verify    # engine and OpenSpiel must agree
```

The `verify` output is itself the first lesson: never trust a tool you haven't checked against an independent one.

The thread through all six weeks: equilibrium guarantees nothing with three players, so exploiting opponents safely needs new tools, and measuring "safely" needs more care than it first appears.

Week 1 — Games, best responses, and equilibrium

By the end of the week, students can compute a best response, say what an equilibrium guarantees in a two-player zero-sum game, and show with a three-line example why that guarantee disappears with three players.

Concepts: extensive-form games and information sets; three-player Leduc (8 cards, 25,800 information sets); best response; Nash equilibrium; exploitability and `nash_conv`; the game value.

Worked example 1.1 — A best response

In rock-paper-scissors (win +1, lose -1, tie 0), the opponent plays Rock 0.5, Paper 0.25, Scissors 0.25. What should we play, and what do we earn?

Expected value of each of our actions:

- Rock: $0(0.5) - 1(0.25) + 1(0.25) = 0$
- Paper: $1(0.5) + 0(0.25) - 1(0.25) = +0.25$
- Scissors: $-1(0.5) + 1(0.25) + 0(0.25) = -0.25$

The best response is pure Paper, earning +0.25. That +0.25 is also how *exploitable* the opponent's strategy is. Uniform play earns 0 against everything and is exploitable by 0: it is the equilibrium, and 0 is the **game value**.

Point to make: in two-player zero-sum games, playing the equilibrium guarantees at least the game value, whatever the opponent does.

Worked example 1.2 — Three players break the guarantee

Players A, B and C. Only B moves, choosing Left or Right. B's payoff is 0 either way; Left gives A +1 and C -1, Right gives A -1 and C +1.

- Every strategy of B is a best response, since B gets 0 regardless.
- So every profile is an equilibrium — and A's payoff can be anything from -1 to +1.
- A cannot do anything about it. There is no single "value" of the game for A.

Point to make: with three players, an opponent with nothing at stake can decide which of the other two pays. The paper calls this the *kingmaker* effect and measures it in Leduc (Week 3 and Week 6).

Worked example 1.3 — Reading nash_conv

Our CFR+ solution has nash_conv 0.0023: summed over the three players, no one can gain more than 0.0023 chips/hand by deviating *alone*.

Ask the students what that does **not** say:

- Nothing about two players deviating at once.
- Nothing about what happens to *us* when someone else deviates — example 1.2 shows it can be anything.

Hands-on

Run `python tree_engine.py verify` and confirm the three seat values match OpenSpiel: -0.0949 , -0.0089 , $+0.1038$. Then have each student write two sentences on why seat 2, acting last, earns the most.

Reading: Shoham & Leyton-Brown, Ch. 3 and 5; paper §1 and §3.1–3.2.

Check questions:

1. In example 1.1, the opponent shifts to Rock 0.4, Paper 0.3, Scissors 0.3. What is the new best response and its value? (*Paper, +0.1.*)
2. Why can't player A in example 1.2 protect itself by changing strategy?
3. Give one situation in real multiplayer poker that resembles example 1.2.

Week 2 — Solving the game: CFR and CFR+

By the end of the week, students can run regret matching by hand, compute a counterfactual value at an information set, and explain what CFR+ changes and why it matters.

Concepts: regret; regret matching; counterfactual values; the average strategy; CFR+ (regret flooring, linear averaging); no convergence guarantee to Nash with three players.

Worked example 2.1 — Regret matching by hand

We play rock-paper-scissors against an opponent who always plays Rock. We start uniform. Each round, **regret** for an action = what it would have earned minus what we actually earned in expectation. The next strategy is proportional to positive cumulative regret.

round	our strategy (R, P, S)	action values vs Rock	our EV	cumulative regret (R, P, S)
1	(1/3, 1/3, 1/3)	(0, +1, -1)	0	(0, +1, -1)
2	(0, 1, 0)	(0, +1, -1)	+1	(-1, +1, -3)
3	(0, 1, 0)	(0, +1, -1)	+1	(-2, +1, -5)

After one round, only Paper has positive regret, so regret matching plays pure Paper — the best response. Against a *fixed* opponent, regret matching finds the best response; when everyone runs it at once, the *average* strategy approaches equilibrium (in two-player zero-sum games).

Worked example 2.2 — A counterfactual value

We are at an information set I containing two histories we can't tell apart (the opponents hold different cards). Weight each history by how likely the **other players and chance** were to reach it — not us:

history	reach by others	value if we play a	value if we play b
h1	0.3	+1	-1
h2	0.1	-2	+2

Counterfactual values: $\text{cfv}(a) = 0.3(+1) + 0.1(-2) = \mathbf{+0.1}$; $\text{cfv}(b) = 0.3(-1) + 0.1(+2) = \mathbf{-0.1}$.

Playing 50/50 now gives $\text{cfv}(I) = 0$, so the regrets are +0.1 for a and -0.1 for b, and the next strategy at I is pure a.

Point to make: our own reach is left out, so every information set learns as if we had tried to get there. That is what lets CFR solve each information set separately.

Worked example 2.3 — What CFR+ changes

Cumulative regrets are (+2, -3, +1). The next round's regrets are (-1, +2, 0).

- **Plain regret matching** adds them: (+1, -1, +1) → next strategy (0.5, 0, 0.5). The middle action just did well but is still buried by its past.
- **CFR+** floors regrets at zero before and after adding: (2, 0, 1) + (-1, 2, 0) = (1, 2, 1) → next strategy (0.25, 0.5, 0.25).

Point to make: CFR+ forgets bad history, so it reacts faster. It also averages strategies with weights proportional to the iteration number.

Worked example 2.4 — Why rare spots converge slowly

In example 2.2, scale both reach probabilities down 100×. The regrets shrink 100× too. An information set reached 1% as often learns about 100× more slowly.

Point to make: play in rarely reached spots is the least settled part of any CFR solution. Week 6 shows this is exactly where safety measurements are fragile.

Hands-on

1. Run `python solve_cfr.py 40 test.pkl` and plot `nash_conv` every 10 iterations. Compare with the 5,000-iteration table's 0.0023.
2. Read `tree_engine.py`: find the forward pass (reach probabilities) and the backward pass (values), and match them to example 2.2.

Reading: Neller & Lanctot, *An Introduction to Counterfactual Regret Minimization* (2013); Zinkevich et al. (2008); Tammelin (2014).

Check questions:

1. In example 2.1, what happens if the opponent switches to Paper at round 4? How many rounds until our strategy changes? (*Our EV stays 0 while Scissors' regret rises by 1 per round from -5, so after 6 rounds it turns positive. With CFR+ flooring, Scissors' regret sat at 0, so it turns positive after one round — example 2.3 in action.*)
2. Why does CFR exclude our own reach from the counterfactual value?
3. With three players, CFR+ still produces a strategy with small `nash_conv`. What is *not* guaranteed?

Week 3 — Biased opponents and what their mistakes are worth

By the end of the week, students can build a biased opponent, explain why α means different things for different types, and trace where an opponent's losses actually go in a three-player game.

Concepts: α -tilted opponents (R over-raiser, C over-caller, F over-folder, O equilibrium); override rates and the equal-deviation axis; best-response gain; an equilibrium player *losing* to deviators; money flowing to the third player (Morton's theorem); kingmakers in real Leduc.

Worked example 3.1 — Building an over-raiser

The tilt: $\sigma'(a) = (1 - \alpha)\sigma^*(a) + \alpha \cdot [a = \text{favoured}]$, applied wherever the favoured action is legal.

At one information set the equilibrium plays raise 0.4, call 0.1, fold 0.5. With $\alpha = 0.35$ toward raising:

- raise: $0.65(0.4) + 0.35 = \mathbf{0.61}$
- call: $0.65(0.1) = \mathbf{0.065}$
- fold: $0.65(0.5) = \mathbf{0.325}$

How often is the equilibrium's choice actually *overridden*? With probability α the action is forced to raise, and that only changes anything when the equilibrium wouldn't have raised: $0.35 \times 0.6 = \mathbf{0.21}$ per decision here.

Worked example 3.2 — Why α isn't comparable across types

Per unit of α , an over-raiser overrides 1.41 decisions per hand, an over-caller 0.89, an over-folder only 0.28. Raising is illegal once betting is capped; folding is illegal with no bet to face, and the equilibrium already folds often.

To produce the same deviation — 0.10 overridden decisions per hand — each type needs a different α :

type	α needed = $0.10 \div \text{rate}$	best-response gain at that deviation (chips/hand)
over-raiser	0.07	+0.10
over-caller	0.11	+0.16
over-folder	0.36	+0.98

Point to make: per unit of *actual deviation*, folding errors are about 10× more valuable than raising errors. Compared at equal α they look the opposite — always check what a parameter really measures.

Worked example 3.3 — Where the money goes

We play equilibrium in seat 0, seat 1 over-raises ($\alpha = 0.35$), seat 2 plays equilibrium. Change in each player's payoff versus all-equilibrium play:

raiser (seat 1)	us (seat 0)	other opponent (seat 2)
-0.329	+0.126	+0.203

The three sum to zero, as they must. But the third player collects more of the raiser's mistake than we do. This is Morton's theorem, known to poker players, measured here exactly. Adapting to the raiser is how we claim a bigger share.

Worked example 3.4 — Equilibrium can lose

Against an extreme over-raiser and over-caller ($\alpha = 1$), playing our equilibrium strategy *loses* 0.692 chips/hand relative to equilibrium play. In a two-player zero-sum game this is impossible: an opponent's deviation can only help an equilibrium player. Ask students which week-1 example explains it.

Worked example 3.5 — Kingmakers in real Leduc

Everyone plays equilibrium; seat 2 switches to a best response. It gains under 0.001 — it is nearly indifferent — yet by choosing among responses within 0.001 chips of its best at each decision, it can move seat 1's payoff anywhere from -0.23 to +0.11. Example 1.2, measured.

Hands-on

Reproduce example 3.3 with the engine (from `rnr_package`):

```
import pickle
from tree_engine import Tree
from q1_alpha_sweep import perturb
T = Tree()
base = pickle.load(open("cfr_3p_5000.pkl", "rb"))["table"]
S = [T.from_table(base, q) for q in range(3)]
eq = T.ev(S)
for a in (0.1, 0.35, 1.0):
    R1 = T.from_table(perturb(base, "R", a), 1)
    print(a, T.ev([S[0], R1, S[2]]) - eq)
```

Then run `python q6_equal_tv.py --cfr cfr_3p_5000.pkl --m 0.1` (about 10 minutes) to reproduce example 3.2.

Reading: paper §3.3, §4.1–4.3, §5; Abou Risk & Szafron (2010).

Check questions:

1. In example 3.1, what is the override rate if the tilt favours folding instead? ($0.35 \times (1 - 0.5) = 0.175$.)
2. At $\alpha = 0.1$ the raiser loses 0.086, we gain 0.023, the other opponent 0.063. Who benefits more from the raiser at small bias?
3. Why can an equilibrium player lose money in example 3.4 but never in rock-paper-scissors?

Week 4 — Identifying opponents

By the end of the week, students can update a posterior over opponent types from observed actions, handle cards they never see, and estimate how many hands identification should take.

Concepts: Bayes' rule over hypotheses; likelihood ratios; marginalizing over hidden cards; accumulating evidence in log-odds; showdown censoring.

Worked example 4.1 — Which actions are informative?

At an information set, the equilibrium plays raise 0.4, call 0.1, fold 0.5. At $\alpha = 0.35$:

- An over-caller calls with probability $0.65(0.1) + 0.35 = 0.415$. A call is $0.415 \div 0.1 = 4.15\times$ more likely from an over-caller than from an equilibrium player.
- An over-raiser raises with probability $0.65(0.4) + 0.35 = 0.61$. A raise is only $0.61 \div 0.4 = 1.53\times$ more likely.

Point to make: the same α produces far more evidence when the favoured action is *rare* in equilibrium. Rare actions move beliefs.

Worked example 4.2 — One Bayes update

Two hypotheses, equally likely: the opponent is an equilibrium player (O) or an over-caller (C). We see one call at that information set.

$$P(C \mid \text{call}) = 0.415 \times 0.5 \div (0.415 \times 0.5 + 0.1 \times 0.5) = \mathbf{0.81}.$$

One rare action moved us from 50% to 81%.

Worked example 4.3 — Cards we never see

The opponent's call depends on its card, which we don't see. Suppose it holds J or Q with equal probability, and the equilibrium calls with probability 0.1 holding J and 0.6 holding Q. Average over the cards:

- Equilibrium: $P(\text{call}) = 0.5(0.1) + 0.5(0.6) = \mathbf{0.35}$
- Over-caller ($\alpha = 0.35$): $P(\text{call}) = 0.5(0.415) + 0.5(0.74) = \mathbf{0.5775}$
- Likelihood ratio: $0.5775 \div 0.35 = \mathbf{1.65}$

Point to make: hiding the card blurs the evidence (1.65 here, versus 4.15 when the equilibrium's call was rare). The paper's identifier does exactly this averaging at every decision, which is why it needs no privileged information. In a full hand, the card probabilities are themselves updated by earlier actions.

Worked example 4.4 — How many hands?

With 16 equally likely pair hypotheses, the true pair starts at odds 1 : 15. Reaching 90% means odds 9 : 1 — a factor of 135, or $\ln 135 \approx \mathbf{4.9 \text{ units of log-evidence}}$. Roughly, each hand adds the log of the likelihood ratio between the true pair and its closest rivals.

Observed medians (hands until the posterior on the true pair reaches 0.9, actions only): 22 hands at $\alpha = 0.35$, so about 0.22 units per hand; 112 hands at $\alpha = 0.1$, about 0.044 per hand.

α	actions only	+ showdown cards	+ all cards revealed
0.10	112	108	54
0.20	46	45	31
0.35	22	19	14

Worked example 4.5 — Why showdowns help so little

Cards are shown only when a hand reaches showdown — and an over-folder, by definition, rarely lets it. The hands we'd most like to see are the ones hidden from us. This is *missing not at random*: the censoring depends on the very behaviour we're trying to measure. The betting itself already carries most of the information, so showdown cards add little.

Hands-on

```
python q2_identify.py --cfr cfr_3p_5000.pkl --alpha 0.35 --pairs 00 R0 C0 F0
--sessions 20 --hands 150
```

Compare the actions-only and all-cards columns. Then repeat at `--alpha 0.1`.

Reading: Southey et al., *Bayes' Bluff: Opponent Modelling in Poker* (2005); paper §6.

Check questions:

1. In example 4.2, a second call is observed. What is the posterior now? (*Odds start at 1 : 1, become 4.15 : 1 after one call and $4.15^2 \approx 17.2 : 1$ after two, so the posterior is $17.2 \div 18.2 \approx 0.95$.)*)
2. If evidence per hand grew like α^2 , the $\alpha = 0.1$ median would be about $12\times$ the $\alpha = 0.35$ one. It is only about $5\times$. Why might that be? (*The α^2 rule holds for small α ; at 0.35 the likelihood ratios are far from 1.*)
3. Folding errors are the most valuable to exploit (Week 3). Using example 4.5, explain why they are also slow to detect.

Week 5 — Safe exploitation: restricted Nash responses

By the end of the week, students can solve a restricted Nash response by hand in rock-paper-scissors, explain why it beats mixing equilibrium with a best response, and describe the three-player version the paper uses.

Concepts: the exploitation–exploitability tradeoff; linear mixing; the restricted Nash response (RNR) as a modified game; parameter p ; the three-player symmetric version with self-interested free opponents; why one-sided trust fails.

Worked example 5.1 — Linear mixing in rock-paper-scissors

Our model says the opponent always plays Rock. Two scores for any strategy of ours:

- **gain** = what we earn if the model is right (against Rock): $P(\text{Paper}) - P(\text{Scissors})$
- **worst case** = what we earn against the opponent's best response to us

Linear mixing plays $(1 - w) \times \text{uniform} + w \times \text{Paper}$:

w	strategy (R, P, S)	gain	worst case
0	(1/3, 1/3, 1/3)	0	0

w	strategy (R, P, S)	gain	worst case
1/3	(2/9, 5/9, 2/9)	+1/3	-1/3
2/3	(1/9, 7/9, 1/9)	+2/3	-2/3
1	(0, 1, 0)	+1	-1

Every unit of gain costs a full unit of worst case: the opponent simply plays Scissors.

Worked example 5.2 — The restricted Nash response

The RNR imagines the opponent follows the model with probability p and otherwise best-responds to us, and maximizes

$$p \cdot \text{gain}(\sigma) + (1-p) \cdot \text{worst}(\sigma)$$

Step 1 — the best possible worst case for each gain. For gain $g \leq 2/3$, the strategy $(1/3, 1/3 + g/2, 1/3 - g/2)$ spreads the risk evenly across Paper and Scissors counterattacks, giving worst case $-g/2$ — half what linear mixing pays. Beyond $g = 2/3$, Scissors is already at 0 and the worst case falls twice as fast, reaching -1 at pure Paper.

Step 2 — maximize along that frontier. The objective is a straight line in g on each piece, so the optimum always sits at a corner:

p	RNR strategy (R, P, S)	gain	worst case
below 1/3	(1/3, 1/3, 1/3)	0	0
1/3 to 2/3	(1/3, 2/3, 0)	+2/3	-1/3
above 2/3	(0, 1, 0)	+1	-1

At gain 2/3, linear mixing's worst case is -2/3; the RNR's is -1/3. (These values were checked with a linear program.)

Two points to make:

1. The RNR dominates linear mixing: same gain, half the exposure.
2. The RNR **jumps** between strategies as p crosses 1/3 and 2/3. The paper sees an abrupt switch in Leduc between $p = 0.5$ and 0.65 (§7.3). This toy shows one mechanism that produces jumps even with exact solutions — whether it is *the* mechanism in Leduc is untested.

Worked example 5.3 — Three players

Now each opponent, independently, follows its biased model with probability p and is otherwise **free**: it knows its mode and maximizes its *own* payoff. We never see the modes. At $p = 0.5$ there are four equally likely worlds:

seat 1	seat 2	probability
follows model	follows model	0.25
free	follows model	0.25
follows model	free	0.25
free	free	0.25

CFR+ is run on this bigger game; our average strategy is the RNR. Why self-interested, not adversarial? If both free opponents minimized *our* payoff in the same hand, they would be acting as a coalition — collusion, which the setting excludes.

What $p = 0$ means: both opponents are always free, so the modified game *is* the real game. RNR at $p = 0$ is an independent equilibrium solve — not “play CFR.”

Worked example 5.4 — The result in Leduc

FF (two over-folders, $\alpha = 0.35$), us in seat 0. Gain and worst case relative to equilibrium; worst case with a pessimistic punisher (Week 6):

strategy	gain	worst case
best equilibrium-level baseline	+0.171	−0.046
RNR $p = 0.25$	+0.299	+0.123
RNR $p = 0.5$	+0.575	+0.039
linear mixing, $w = 0.5$	+0.951	−1.059
RNR $p = 0.9$	+1.708	−0.531
best response	+1.846	−3.860

RNR at $p = 0.5$ more than triples the profit while staying *harder* to punish than equilibrium. Across the paper, safe exploitation holds in 41 of 45 settings at $\alpha = 0.35$.

Worked example 5.5 — One-sided trust fails

An earlier version restricted only seat 1 and trusted seat 2's model completely. At $p = 0.75$ it earned +1.687 — but when seat 2 switched to punishing, the worst case was **−3.595**, nearly as bad as a pure best response. The original check only let seat 1 punish, so the failure went unseen for a full draft. Lesson: test every opponent you are trusting.

Hands-on

1. A short solve (writes a separate file, about 2–5 minutes):

```
python rnr_general.py --pair FF --alpha 0.35 --seat 0 --p 0.5 --restrict
independent --free selfish --iters 60 --suffix short
```

Compare its gain and worst case with

`results/rnr_FF_a0.35_s0_independent_selfish_p0.5.json` (600 iterations).

2. Plot the frontier from saved results:

```
import json, matplotlib.pyplot as plt
r = json.load(open("results/pess/pess_FF_a0.35_s0.json"))
lin = [r["linear"][w] for w in sorted(r["linear"], key=float)]
rnr = [r["rnr"][p] for p in sorted(r["rnr"], key=float)]
plt.plot([x["worst"] for x in lin], [x["gain"] for x in lin], "o-",
label="linear mixing")
plt.plot([x["worst"] for x in rnr], [x["gain"] for x in rnr], "s-",
label="restricted response")
plt.xlabel("worst case vs equilibrium"); plt.ylabel("gain vs equilibrium")
plt.legend(); plt.show()
```

Reading: Johanson, Zinkevich & Bowling, *Computing Robust Counter-Strategies* (2007); paper §7.1–7.5; optional: Ganzfried, *Safe Equilibrium* (2023).

Check questions:

1. In example 5.2 at $p = 0.5$, compute the objective for the three corner strategies. Which wins? (*Uniform 0; $(1/3, 2/3, 0)$ gives $0.5(2/3) + 0.5(-1/3) = 1/6$; pure Paper gives 0. The middle one.*)
2. Why must the free opponents in example 5.3 be self-interested rather than adversarial?
3. Explain to a classmate why " $p = 0$ " is not the same as "play our CFR strategy."

Week 6 — Measuring safety honestly

By the end of the week, students can say exactly what a "worst case" number measures, when it can be trusted, and how each claim in the paper was tested. The meeting is built around real mistakes made — and caught — during this project.

Concepts: self-interested vs. adversarial punishers; tie-breaking and the pessimistic punisher; solution noise and materiality thresholds; choice of baseline; both opponents adapting; robustness to model errors.

Worked example 6.1 — Who is the punisher?

Return to example 1.2 (B chooses who pays; B gets 0 either way).

- A **self-interested** B maximizes its own payoff. It is indifferent, so any choice is a "best response."
- An **adversarial** B minimizes A's payoff and always plays Right: A gets -1 .

An adversary measures *spite* — it will give up money to hurt us, and with three players that money goes to someone else. The paper uses self-interested punishers as its main measure and reports adversarial ones only as a bound.

Worked example 6.2 — Tie-breaking decides who pays

A best-response program, meeting a tie, picks the first action — Left — so A appears to get $+1$. A **pessimistic** punisher picks, among responses within ϵ of its best, the one worst for us — Right — so A gets -1 . Same game, opposite answers.

In Leduc with everyone at equilibrium, the reported value for seat 1 was -0.080 ; the true range was -0.23 to $+0.11$. With a *biased* third player at the table the punisher usually has a clear best response, and the pessimistic value changes nothing in most settings — but by up to 0.24 in a few, so the paper now uses it everywhere.

Worked example 6.3 — Is a difference real?

Every number is an exact evaluation of the strategy computed, but the strategy is an approximate solution. Re-solving six settings with twice the iterations, or from a random start, changed gain by at most 0.010 and worst case by at most 0.030 .

Decide which claims stand (materiality threshold 0.05):

claim	margin	verdict
RNR is safer than the baseline	+0.02	not established — inside the noise
RNR is safer than the baseline	+0.12	established
RNR earns more than the baseline	+0.40	established

Trap: an earlier estimate put the noise at 0.053 — it had compared against a 90-iteration solve that hadn't converged. Calibrate noise with *converged* solutions.

Worked example 6.4 — Which baseline?

Three equilibrium-level strategies earn the same against FO (to 0.002) but differ in worst case:

strategy	worst case vs equilibrium
CFR, 5,000 iterations	−0.025
CFR, 1,000 iterations (same run, stopped early)	+0.067
RNR at $p = 0$	+0.070

Comparing an adaptive strategy against only the first would credit adaptation for what is really a luckier baseline. The paper compares against the best of all three on each axis.

Worked example 6.5 — When both opponents adapt

Hold our equilibrium strategy fixed and let both opponents learn, each maximizing its own payoff. From three different starting points they converge to the same place: **we lose 0.391, seat 1 gains 0.141, seat 2 gains 0.251** per hand. Both opponents prefer it, and neither colluded. RNR at $p = 0.5$ loses only 0.213 in the same test.

In practice we would notice the change and adapt back, so the real cost is roughly *exposure per hand* $\times$ *hands until we notice* — which links this week back to Week 4.

Catch-the-error session

For each case: present the claim, let students propose a check, then reveal what happened.

claim that was made	the check that exposed it	lesson
One-sided RNR is less exploitable than CFR	letting the <i>trusted</i> opponent punish: -3.6	test every opponent you trust
Acting later makes adaptation safer	measuring each seat against its own equilibrium, not raw payoffs	compare on one scale
RNR at $p = 0$ is "a different equilibrium"	CFR's own earlier checkpoint behaved the same way	an explanation needs a test that could have ruled it out
A lone deviator moves our payoff by -0.080	a pessimistic vs. optimistic range: -0.23 to +0.11	a number set by tie-breaking isn't a property of the game
Random-start runs converge 10× slower	seeded regrets outweighed 6,000 real updates	when results look odd, suspect your code first
Some wins are "within convergence error"	evaluations are exact; the question is run-to-run noise	name which uncertainty you mean

Hands-on

1. Reproduce example 6.2 in Leduc (we sit in seat 1; seat 2 deviates):

```
import pickle
from tree_engine import Tree
from reeval_pessimistic import respond_pess
T = Tree(); U = T.util
base = pickle.load(open("cfr_3p_5000.pkl", "rb"))["table"]
S = [T.from_table(base, q) for q in range(3)]; eq = T.ev(S)
prof = [S[0], S[1], None]
_, exact = T.respond(prof, 2, U[:, 2], U[:, 1])
lo = respond_pess(T, prof, 2, U[:, 2], U[:, 1], 0.001)
hi = -respond_pess(T, prof, 2, U[:, 2], -U[:, 1], 0.001)
print("exact", exact - eq[1], "range", lo - eq[1], hi - eq[1])
```

2. `python gaps_check.py misspec` — how do strategies tuned for FF fare against other opponents?
3. `python gaps_check.py bothadapt --iters 100 --strategies CFR --report-every 50` — watch our payoff fall as the opponents learn.

Reading: paper §3.4, §4.3–4.5, §7.6–7.9, §8.

Deliverable: a 10-minute presentation per student — one claim from the paper, how it was verified, and what result would have falsified it.

Check questions:

1. Why is "our payoff when an opponent best-responds" ill-defined when that opponent is indifferent?
2. A classmate reports a new strategy that is 0.03 safer than the baseline. What do you ask before believing it?
3. In example 6.5, if detecting the opponents' change takes 20 hands, roughly what does playing CFR cost compared with RNR at $p = 0.5$? (*About $20 \times (0.391 - 0.213) \approx 3.6$ chips.*)

Compressing to 3–5 weeks

Every version keeps the catch-the-error session: it teaches more research practice per minute than anything else in the plan.

weeks available	schedule	what to cut
5	1 · 2 · 3 · 4 · 5 + 6 combined	Week 6: examples 6.4 and 6.5; keep 6.1–6.3 and the error session
4	1 + 2 · 3 · 4 · 5 + 6	Week 2: example 2.4 and the tree-engine reading; Week 6 as in the 5-week version
3	1 + 2 · 3 + 4 · 5 + 6	also Week 3: examples 3.2 and 3.5; Week 4: examples 4.3 and 4.5; hands-on runs become instructor demos

In the 3-week version, assign the Week 1 and Week 3 readings before the first meeting so meeting time goes to the worked examples.

Appendix: commands and readings

All commands run from `rnr_package/` with OpenSpiel, NumPy and Matplotlib installed. Times are for one process on a modern desktop; they roughly double when many jobs share the machine.

week	command or snippet	about	reproduces
setup	<code>python tree_engine.py build</code>	2 min	the flattened game tree
1	<code>python tree_engine.py verify</code>	1 min	seat values; engine matches OpenSpiel
2	<code>python solve_cfr.py 40 test.pkl</code>	3 min	nash_conv falling with iterations
3	money-flow snippet (Week 3)	1 min	example 3.3
3	<code>python q6_equal_tv.py --cfr cfr_3p_5000.pkl --m 0.1</code>	10 min	example 3.2
4	<code>python q2_identify.py --cfr cfr_3p_5000.pkl --alpha 0.35 --pairs 00 R0 C0 F0 --sessions 20 --hands 150</code>	2 min	identification times
5	<code>python rnr_general.py ... --iters 60 --suffix short</code> (Week 5)	2–5 min	a quick restricted response
5	frontier-plot snippet (Week 5)	seconds	the RNR vs. linear frontier
6	kingmaker snippet (Week 6)	1 min	−0.080 and the −0.23 to +0.11 range
6	<code>python gaps_check.py misspec</code>	2 min	robustness to model errors
6	<code>python gaps_check.py bothadapt --iters 100 --strategies CFR --report-every 50</code>	5 min	the second equilibrium emerging

One caveat: `tree_engine.py verify` reads `cfr_3p_1000.pkl`. If that file holds the original 1,000-iteration table, the seat values will be −0.0936, −0.0087, +0.1023 rather than the paper's; both are correct for their table.

Readings

- Shoham, Y. & Leyton-Brown, K. (2009). *Multiagent Systems*. Ch. 3 and 5. — Week 1
- Neller, T. & Lanctot, M. (2013). *An Introduction to Counterfactual Regret Minimization*. — Week 2

- Zinkevich, M., Johanson, M., Bowling, M. & Piccione, C. (2008). Regret minimization in games with incomplete information. *NIPS 20*. — Week 2
- Tammelin, O. (2014). Solving large imperfect information games using CFR+. arXiv:1407.5042. — Week 2
- Abou Risk, N. & Szafron, D. (2010). Using counterfactual regret minimization to create competitive multiplayer poker agents. *AAMAS*. — Week 3
- Southey, F. et al. (2005). Bayes' bluff: Opponent modelling in poker. *UAI*. — Week 4
- Johanson, M., Zinkevich, M. & Bowling, M. (2007). Computing robust counter-strategies. *NIPS 20*. — Week 5
- Ganzfried, S. (2023). Safe equilibrium. arXiv:2201.04266. — Week 5, optional
- The paper, draft v9: sections as listed in each week.